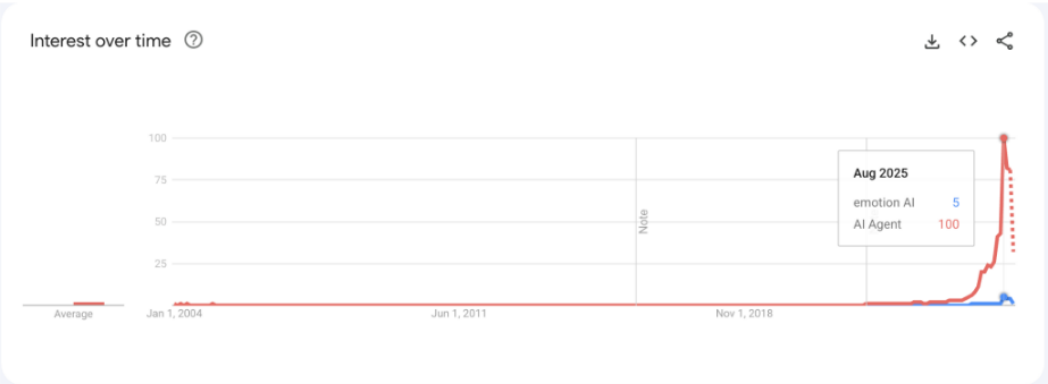
Lesson 43: Intro to LLM, ChatGPT, Agent, MCP, OpenAI API and Langchain

History of ChatGPT



Early Symbolic Agents (1950s–1970s)

Early Learning Systems (1960s–1980s)

Foundations of Modern Reinforcement Learning (1980s)

Classical RL Algorithms (Late 1980s–1990s)

RL in Robotics and Control (1990s–2000s)

Deep Learning Meets RL (2010–2015)

Deep RL Breakthroughs (2015–2020)

Multi-Agent RL and Emergence (2017–Present)

Memory-Augmented and Cognitive Agents (2020–Present)

LLM-Based Agentic Systems (2022–Present)

AI Agent

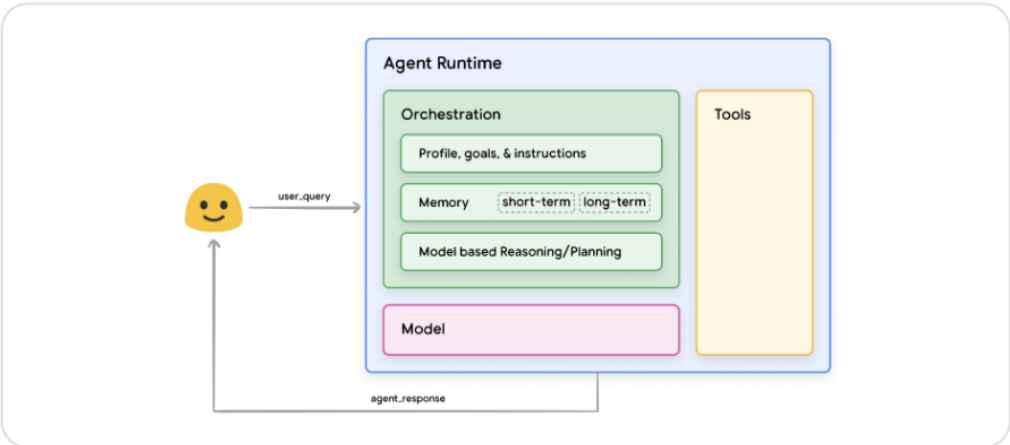

Agent Runtime
Orchestration
Profile, goals, & instructions
Memory
short-term
long-term
Model based Reasoning/Planning
Tools
Model
user_query
agent_response

Figure 1. General agent architecture and components

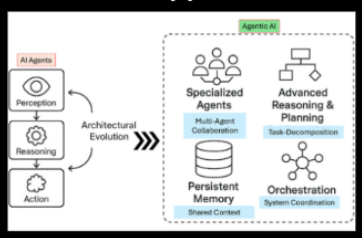
Agents. (2025). Kaggle.com. https://www.kaggle.com/whitepaper-agents

Taxonomy of AAA: AI Agent vs. Agentic AI vs. Agent AI

"The combination of reasoning, logic, and access to external information that are all connected to a Generative AI model invokes the concept of an agent." [1]

"An LLM agent runs tools in a loop to achieve a goal." [2]

"Agentic AI is an autonomous AI system that can act independently to achieve pre-determined goals." [3]



[4]

"We define "Agent AI" as a class of interactive systems that can perceive visual stimuli, language inputs, and other environmentally grounded data, and can produce meaningful embodied actions" [5]

References
[1] Agents. (2025). Kaggle.com. https://www.kaggle.com/whitepaper-agents
[2] Willison, S. (2025). I think "agent" may finally have a widely enough agreed upon definition to be useful jargon now. Simon Willison's Weblog. https://simonwillison.net/2025/Sep/18/agents/
[3] "What Is Agentic AI? Agentic AI Explained - AWS." Amazon Web Services, Inc., 25 July 2025, aws.amazon.com/what-is/agentic-ai/.
[4] Sapkota, R., Roumeliotis, K. I., & Karkee, M. (2025). AI Agents vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenge. ArXiv.org. https://arxiv.org/abs/2505.10468
[5] Durante, Zane, et al. "Agent AI: Surveying the Horizons of Multimodal Interaction." ArXiv.org, 25 Jan. 2024, arxiv.org/abs/2401.03568.

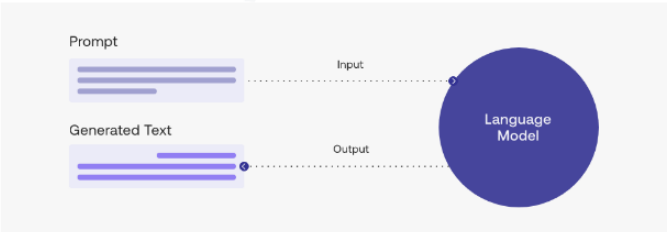
History of ChatGPT

What is it… GPT - "Generative Pre-trained Transformer"

An AI language model trained on a massive amount of text data that generates natural language responses in a conversational style.

- GPT-1 was trained with BooksCorpus dataset (5GB), whose primary focus was language understanding.
- On Valentine's Day 2019, GPT-2 was launched with the slogan "too dangerous to release." It was trained with Reddit articles with over 3 likes (40GB). The training cost was $43,000.
- In June 2020, GPT-3 was trained on a massive corpus of text data, around 570GB of datasets, including books, webtexts, Wikipedia, articles and other pieces of writing on the internet. To be even more exact, 300 billion words were fed into the system.
- ChatGPT was launched on Nov 30 2022. This AI model costs an estimated $12 million to train for a single training run.
- GPT-3 model is open sourced. However, ChatGPT's model is not. LLAMA3 is an open sourced LLM from Meta.
- Token:
    - Tokens can be thought of as pieces of words. For English text, 1 token is approximately 4 characters or 0.75 words. https://platform.openai.com/tokenizer
    - token limits - The token limit for gpt-35-turbo is 4096 tokens, whereas the token limits for gpt-4 and gpt-4-32k are 8192 and 32768 respectively. Now, gpt-4-turbo supports 128K tokens. These limits include the token count from both the message array sent and the model response.

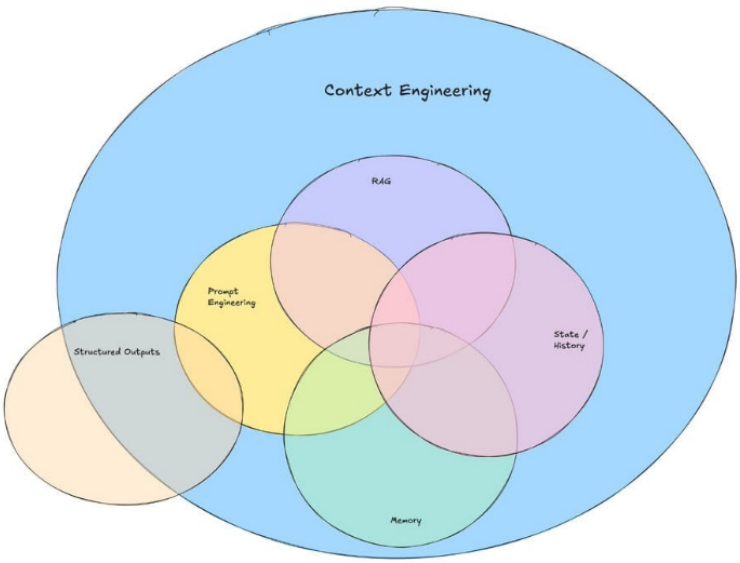
Prompt Engineering → Context Engineering (2025)

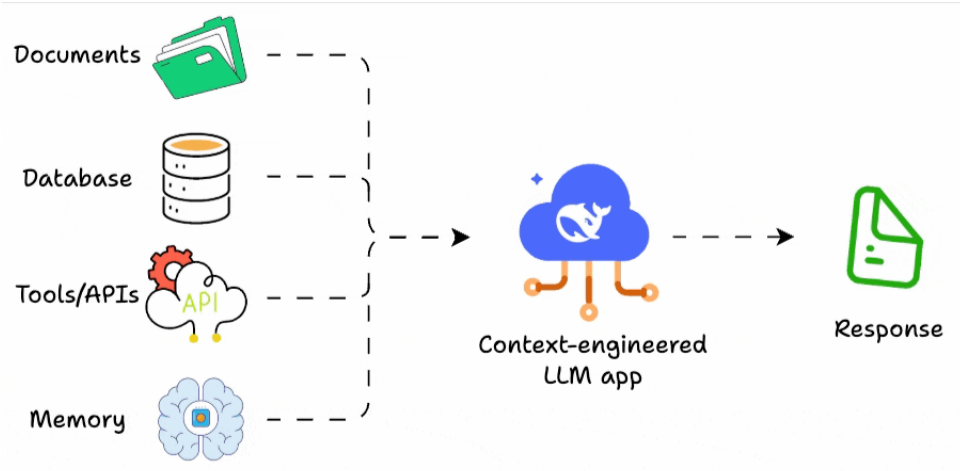


What is Prompt Engineering

Prompt engineering is the process of designing and optimizing prompts used in natural language processing (NLP) models, such as ChatGPT, chatbots, or virtual assistants. This involves crafting prompts that are clear, concise, and effective in eliciting the desired response. (Aka prompt engineering is like making an effective fishing lure - just as a well-designed lure is more likely to catch the fish you want, a well-crafted prompt is more likely to elicit the desired response.)
https://platform.openai.com/docs/guides/prompt-engineering

Context Engineering





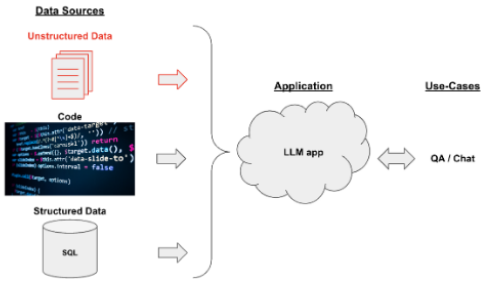
Langchain

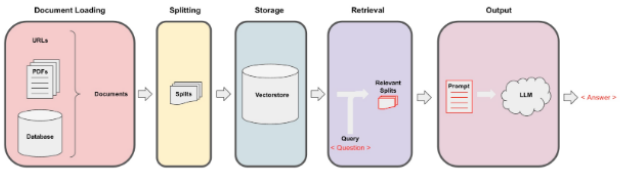
https://www.langchain.com/

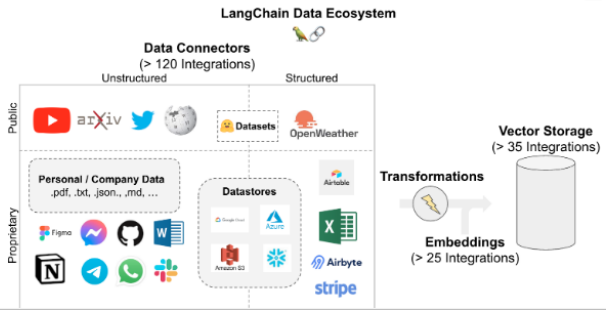
LangChain Introduction

LangChain is a framework for developing applications powered by language models. It enables applications that are:

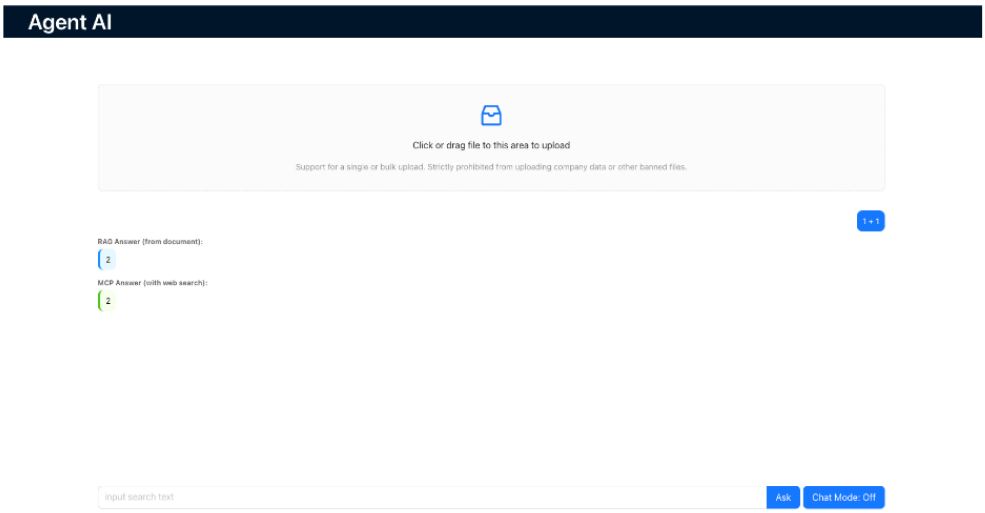
- Data-aware: connect a language model to other sources of data
- Agentic: allow a language model to interact with its environment


Data Sources
Unstructured Data
Code
Structured Data
SQL
Application
LLM app
Use-Cases
QA / Chat


Document Loading
Splitting
Storage
Retrieval
Output
URLs
PDFs
Database
Documents
Splits
Vectorstore
Relevant Splits
Query
< Question >
Prompt
LLM
< Answer >


LangChain Data Ecosystem
Data Connectors
(> 120 Integrations)
Unstructured
Structured
Public
Proprietary
Datasets
OpenWeather
Personal / Company Data
.pdf, .txt, .json., .md, ...
Datastores
Airtable
Airbyte
stripe
Transformations
Vector Storage
(> 35 Integrations)
Embeddings
(> 25 Integrations)

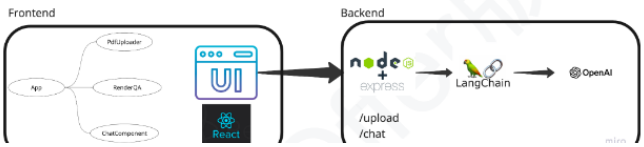
Project Introduction



Features:

- users can upload a PDF
- users can ask questions in the PDF
- users will receive answers on questions asked
- users can ask questions about the PDF using speech
- users can hear the answers in speech
- users can get the results from online search

High level Project Diagram

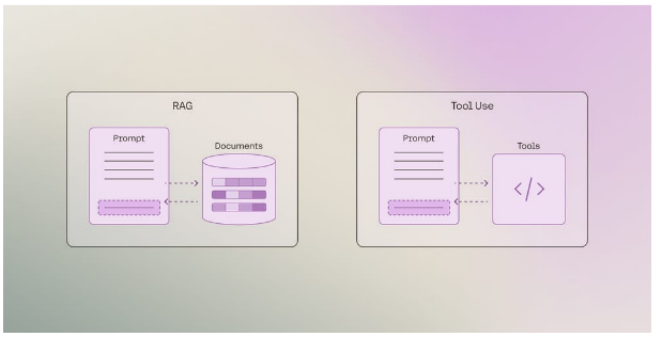


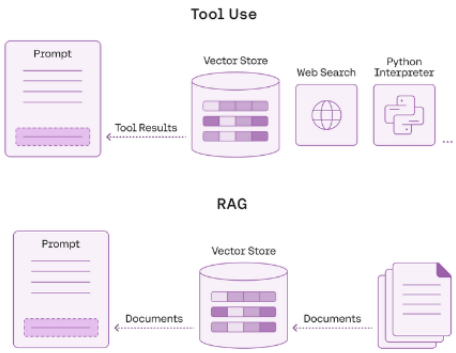
What are Node.js and Express.js

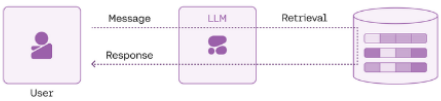
Node.js - Node.js® is an open-source, cross-platform JavaScript runtime environment. https://nodejs.org/en/download

Express.js - web framework for Node.js https://expressjs.com/

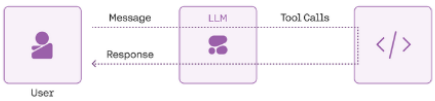
From RAG to Tool Use

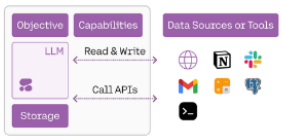




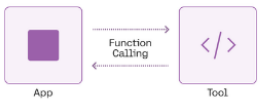


Tool use



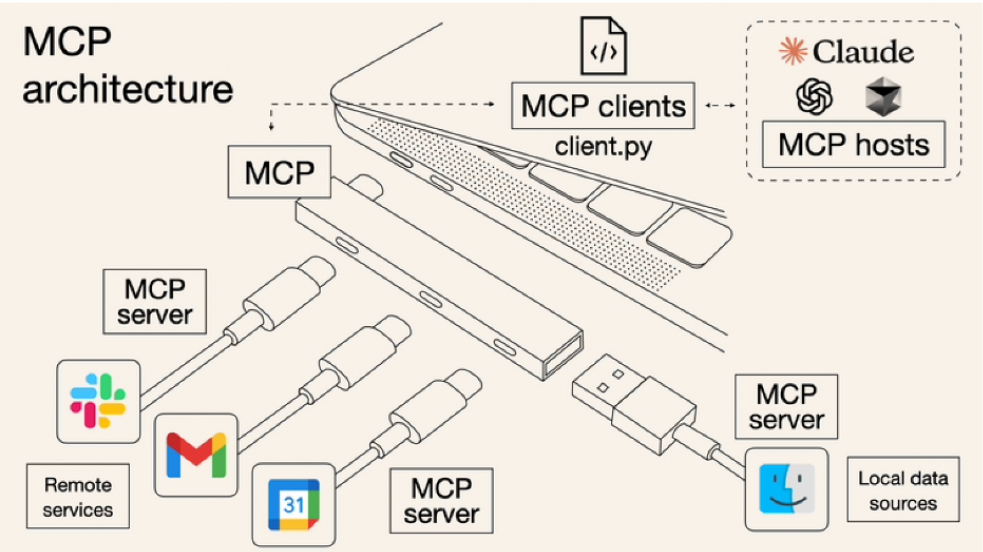


If tool use is the *what* of LLM capability extension, function calling is the *how*.



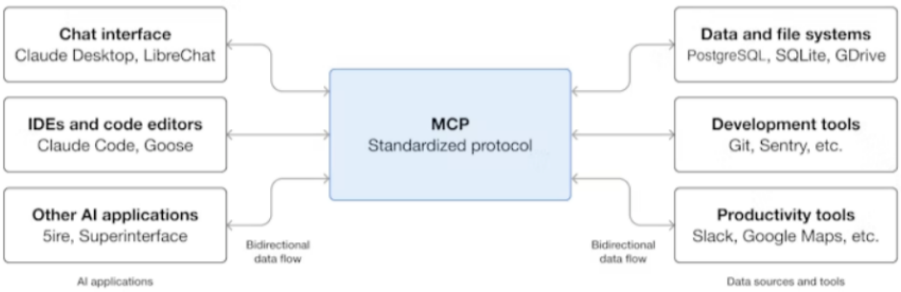
src: https://cohere.com/llmu/from-rag-to-tool-use

Now, let's talk about MCP



src: https://docs.openg2p.org/utilities-and-tools/unified-conversation-agent/uca-research-and-development/modal-context-protocol-mcp

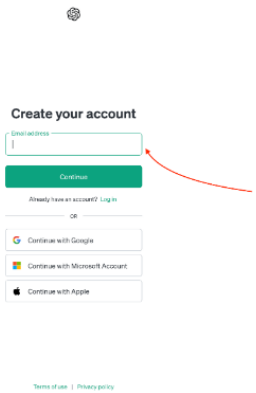
The Model Context Protocol is an open standard that enables developers to build secure, two-way connections between their data sources and AI-powered tools. The architecture is straightforward: developers can either expose their data through MCP servers or build AI applications (MCP clients) that connect to these servers.



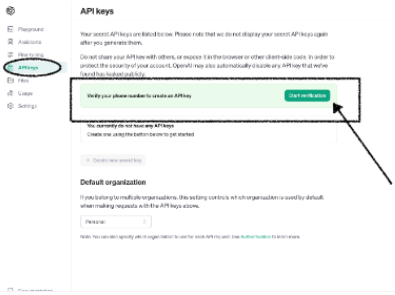
src: https://modelcontextprotocol.io/docs/getting-started/intro

Obtain OpenAI API key (need a US or oversea phone number)

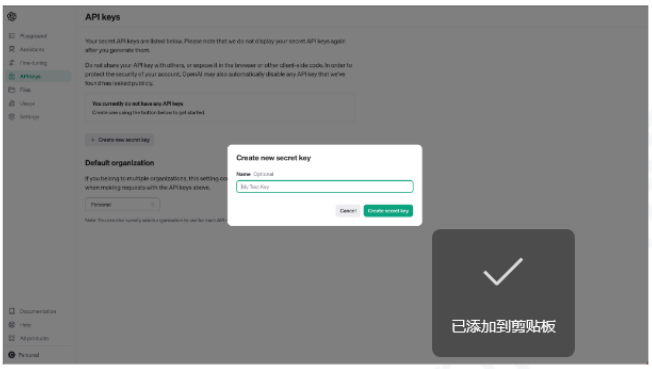
sign up for an OpenAI account https://platform.openai.com/

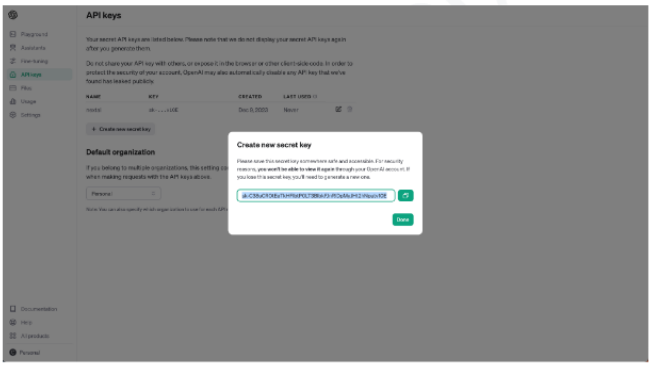


Verify your email
We sent an email to
[redacted]
Click the link inside to get started.
Open Gmail
Resend email
Tell us about you
Full name
Organization name (optional)
Birthday
By clicking "Agree", you agree to our Terms and have read our Privacy Policy.
Agree


API keys
Playground
Assistants
Dashboard
API keys
Files
Logs
Settings
Your secret API keys are listed below. Please note that we do not display your secret API keys again after you generate them.
Do not share your API key with others, or expose it in the browser or other client-side code. In order to protect the security of your account, OpenAI may also automatically disable any API key that we've found has leaked publicly.
Verify your phone number to create an API key
Start verification
You currently do not have any API keys.
Create one using the button below to get started.
+ Create new secret key
Default organization
If you belong to multiple organizations, this setting controls which organization is used by default when making requests with the API keys above.
Personal

https://platform.openai.com/api-keys





How to prepay?

https://platform.openai.com/settings/organization/billing/overview

Project Setup

First, let's set up our React application. In the terminal, run `**npx create-react-app agentai**`

Lesson 46: Build the Express.js backend

Let's build the backend.

Create a new folder called `**server**` under your **agentai** folder (NOTE: it is **NOT** under /src).

We will build our Express.js backend in this `**server**` folder, after it's done, it will look something like this. No rush to create this structure now. We will do it step by step in this class.

```
server/
├── chat.js
├── package-lock.json
├── package.json
├── .env
├── server.js
└── uploads
    ├── hbs-lean-startup.pdf
    └── any-pdf-file-of-your-choice.pdf
```

First, let's create server/package.json file with the following content

```
{
  "name": "server",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  }
```

```
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "@langchain/classic": "^1.0.5",
    "@langchain/community": "^1.0.5",
    "@langchain/core": "^1.1.0",
    "@langchain/openai": "^1.1.3",
    "@modelcontextprotocol/sdk": "^1.23.0",
    "cors": "^2.8.5",
    "dotenv": "^16.3.1",
    "express": "^4.18.2",
    "langchain": "^1.1.1",
    "multer": "^1.4.5-lts.1",
    "pdf-parse": "^1.1.1",
    "serpapi": "^2.2.1",
    "zod": "^4.1.13"
  },
  "type": "module"
}
```

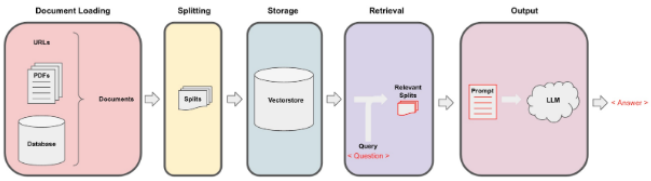
- Run `**npm install --legacy-peer-deps**`, under "/server" path
- Create a folder called `**uploads**` inside the /server, it will save the uploaded files to this folder
- Create a file called **.env** with the following content
  OPENAI_API_KEY=<YOUR_API_KEY>

  e.g.
  OPENAI_API_KEY=sk-ABCDEFG

Next, find the .gitignore file in the root and add server/.env to it

This is the sample file if you ever need it hbs-lean-startup.pdf

Create a file called **chat.js** with the following content




```
import { RecursiveCharacterTextSplitter } from "@langchain/textsplitters";
import { OpenAIEmbeddings } from "@langchain/openai";
import { MemoryVectorStore } from "@langchain/classic/vectorstores/memory";
import { ChatOpenAI } from "@langchain/openai";
import { PromptTemplate } from "@langchain/core/prompts";
import { PDFLoader } from "@langchain/community/document_loaders/fs/pdf";

// NOTE: change this default filePath to any of your default file name
const chat = async (filePath = "./uploads/hbs-lean-startup.pdf", query) => {
  // Get API key from environment
  const apiKey = process.env.OPENAI_API_KEY;

  // step 1:
  const loader = new PDFLoader(filePath);

  const data = await loader.load();

  // step 2:
  const textSplitter = new RecursiveCharacterTextSplitter({
    chunkSize: 500, //  (in terms of number of characters)
    chunkOverlap: 0,
  });

  const splitDocs = await textSplitter.splitDocuments(data);

  // step 3

  const embeddings = new OpenAIEmbeddings(apiKey ? { apiKey } : {});

  const vectorStore = await MemoryVectorStore.fromDocuments(
    splitDocs,
    embeddings
  );
```

来Offer版权所有

```
  // step 4: retrieval

  // const relevantDocs = await vectorStore.similaritySearch(
  // "What is task decomposition?"
  // );

  // step 5: qa w/ customize the prompt
  const model = new ChatOpenAI({
    model: "gpt-5",
    ...(apiKey && { apiKey }),
  });

  const template = `Use the following pieces of context to answer the question at the end.
If you don't know the answer, just say that you don't know, don't try to make up an answer.
Use three sentences maximum and keep the answer as concise as possible.

{context}
Question: {question}
Helpful Answer:`;

  const prompt = PromptTemplate.fromTemplate(template);

  // Use retriever to get relevant documents

  const retriever = vectorStore.asRetriever();
  const relevantDocs = await retriever.invoke(query);

  // Format context from retrieved documents
  const context = relevantDocs.map((doc) => doc.pageContent).join("\n\n");

  // Create a simple chain using the prompt template
  const formattedPrompt = await prompt.format({
    context,
    question: query,
  });

  // Get response from the model
  const response = await model.invoke(formattedPrompt);

  return { text: response.content };
};

export default chat;
```

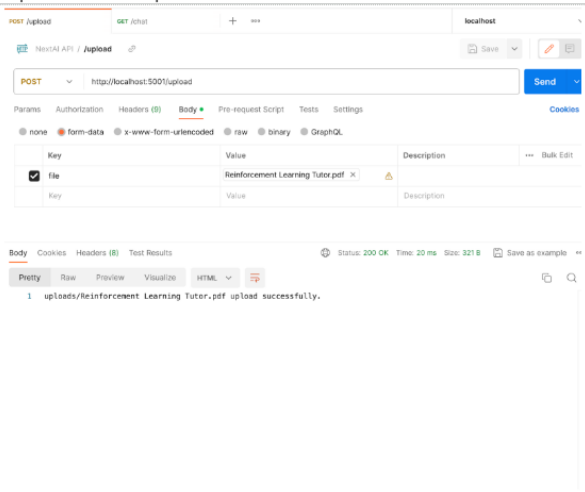
Create a file called **server.js** with the following content

```
import express from "express";
import cors from "cors";
import dotenv from "dotenv";
import multer from "multer"; // Import multer
import chat from "./chat.js";

dotenv.config();

const app = express();
app.use(cors());

// Configure multer
const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, "uploads/");
  },
  filename: function (req, file, cb) {
    cb(null, file.originalname);
  },
});
const upload = multer({ storage: storage });

const PORT = 5001;

let filePath;

app.post("/upload", upload.single("file"), (req, res) => {
  // Use multer to handle file upload
  filePath = req.file.path; // The path where the file is temporarily saved
  res.send(filePath + " upload successfully.");
});

app.get("/chat", async (req, res) => {
  const resp = await chat(filePath, req.query.question); // Use MCP-enhanced chat
  res.send({
    ragAnswer: resp.text,
    mcpAnswer: "N/A",
  });
});

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

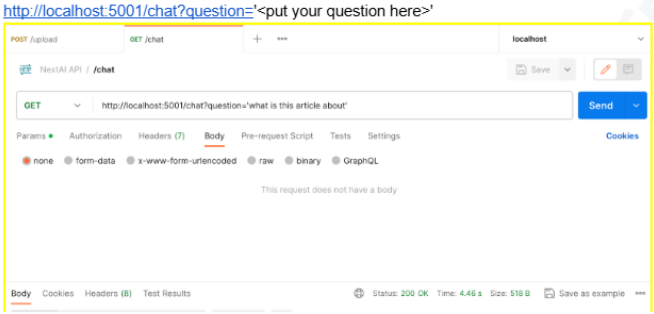
Now, let's run the server (`node server.js`) and use Postman to test our Express.js backend

来Offer版权所有

Test the /upload endpoint
http://localhost:5001/upload

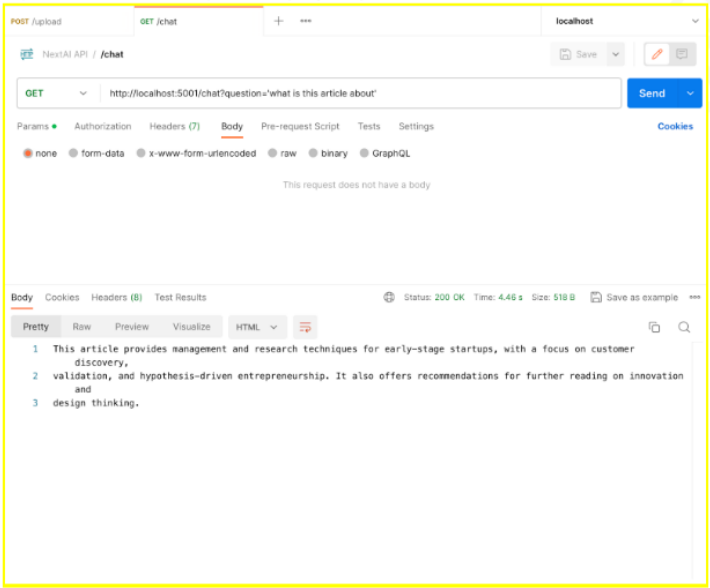


Test the /chat endpoint
http://localhost:5001/chat?question='<put your question here>'

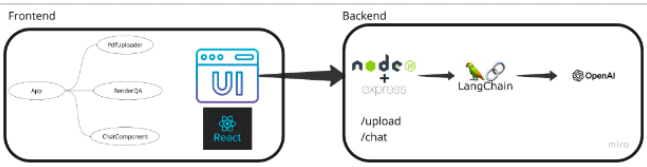


Test the /chat endpoint

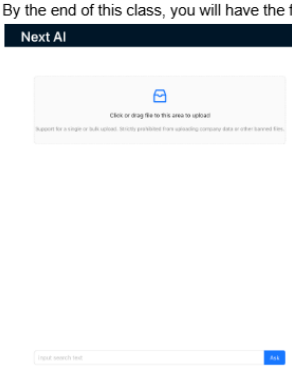
http://localhost:5001/chat?question='<put your question here>'



Lesson 47: Build the React UI



By the end of this class, you will have the following working app.



Under root, update **package.json** with the followings

```
{
"name": "agentai",
"version": "0.1.0",
"private": true,
"dependencies": {
  "@testing-library/jest-dom": "^5.17.0",
  "@testing-library/react": "^13.4.0",
  "@testing-library/user-event": "^13.5.0",
  "antd": "^5.8.4",
  "axios": "^1.5.0",
  "concurrently": "^8.2.1",
  "react": "^18.2.0",
  "react-dom": "^18.2.0",
  "react-scripts": "5.0.1",
  "web-vitals": "^2.1.4"
},
"scripts": {
  "dev": "concurrently \"npm start\" \"npm run server\"",
  "server": "cd server && npm run start",
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "react-scripts test",
  "eject": "react-scripts eject"
},
"eslintConfig": {
"extends": [
  "react-app",
  "react-app/jest"
]
},
"browserslist": {
  "production": [
    ">0.2%",
    "not dead",
    "not op_mini all"
],
```

```
"development": [
      "last 1 chrome version",
      "last 1 firefox version",
      "last 1 safari version"
    ]
```

```
  }
}
```

Next, let's install all dependencies by running `**npm install**`

Under **src,** create a folder called `**components**`

src/components/ChatComponent.js

```
import React, { useState } from "react"; // Import useState
import axios from "axios";
import { Input } from "antd";

const { Search } = Input;

const DOMAIN = "http://localhost:5001";

const searchContainer = {
  display: "flex",
  justifyContent: "center",
};

const ChatComponent = (props) => {
  const { handleResp, isLoading, setIsLoading } = props;
  // Define a state variable to keep track of the search value
  const [searchValue, setSearchValue] = useState("");

  const onSearch = async (question) => {
    // Clear the search input
    setSearchValue("");
    setIsLoading(true);

    try {
      const response = await axios.get(`${DOMAIN}/chat`, {
        params: {
          question,
        },
      });
      handleResp(question, response.data);
    } catch (error) {
      console.error(`Error: ${error}`);
      handleResp(question, error);
    } finally {
      setIsLoading(false);
    }
  };

  const handleChange = (e) => {
    // Update searchValue state when the user types in the input box
    setSearchValue(e.target.value);
  };
```

Offer版权所有

```
  return (
    <div style={searchContainer}>
      <Search
        placeholder="input search text"
        enterButton="Ask"
        size="large"
        onSearch={onSearch}
        loading={isLoading}
        value={searchValue} // Control the value
        onChange={handleChange} // Update the value when changed
      />
    </div>
  );
};

export default ChatComponent;
```

src/components/PdfUploader.js

```
import React from "react";
import axios from "axios"; // Import axios for HTTP requests
import { InboxOutlined } from "@ant-design/icons";
import { message, Upload } from "antd";

const { Dragger } = Upload;

const DOMAIN = "http://localhost:5001";
```

```
const uploadToBackend = async (file) => {
  const formData = new FormData();
  formData.append("file", file);
  try {
    const response = await axios.post(`${DOMAIN}/upload`, formData, {
      headers: {
        "Content-Type": "multipart/form-data",
      },
    });
    return response;
  } catch (error) {
    console.error("Error uploading file: ", error);
    return null;
  }
};
```

```
const attributes = {
  name: "file",
  multiple: true,
  customRequest: async ({ file, onSuccess, onError }) => {
    const response = await uploadToBackend(file);
    if (response && response.status === 200) {
      // Handle success
      onSuccess(response.data);
    } else {
      // Handle error
      onError(new Error("Upload failed"));
    }
  },
  onChange(info) {
    const { status } = info.file;
    if (status !== "uploading") {
      console.log(info.file, info.fileList);
    }
    if (status === "done") {
      message.success(`${info.file.name} file uploaded successfully.`);
    } else if (status === "error") {
      message.error(`${info.file.name} file upload failed.`);
    }
  },
  onDrop(e) {
    console.log("Dropped files", e.dataTransfer.files);
  },
};

const PdfUploader = () => {
  return (
    <Dragger {...attributes}>
      <p className="ant-upload-drag-icon">
        <InboxOutlined />
      </p>
      <p className="ant-upload-text">
        Click or drag file to this area to upload
      </p>
      <p className="ant-upload-hint">
        Support for a single or bulk upload. Strictly prohibited from uploading
        company data or other banned files.
      </p>
    </Dragger>
  );
};

export default PdfUploader;
```

来Offer版权所有

src/components/RenderQA.js

```
import React from "react";
import { Spin } from "antd";

const containerStyle = {
  display: "flex",
  justifyContent: "space-between",
  flexDirection: "column",
  marginBottom: "20px",
};

const userContainer = {
  textAlign: "right",
};

const agentContainer = {
  textAlign: "left",
};

const userStyle = {
```

```
  maxWidth: "50%",
  textAlign: "left",
  backgroundColor: "#1677FF",
  color: "white",
  display: "inline-block",
  borderRadius: "10px",
  padding: "10px",
  marginBottom: "10px",
};

const answerContainer = {
  marginBottom: "10px",
};

const answerLabel = {
  fontSize: "12px",
  fontWeight: "bold",
  color: "#666",
  marginBottom: "5px",
};
```

```
const ragAnswerStyle = {
  maxWidth: "50%",
  textAlign: "left",
  backgroundColor: "#E6F7FF",
  color: "black",
  display: "inline-block",
  borderRadius: "10px",
  padding: "10px",
  marginBottom: "5px",
  borderLeft: "4px solid #1890FF",
};

const mcpAnswerStyle = {
  maxWidth: "50%",
  textAlign: "left",
  backgroundColor: "#F6FFED",
  color: "black",
  display: "inline-block",
  borderRadius: "10px",
  padding: "10px",
  marginBottom: "5px",
  borderLeft: "4px solid #52C41A",
};

const RenderQA = (props) => {
  const { conversation, isLoading } = props;
```

版权所有

```
  return (
    <>
      {conversation?.map((each, index) => {
        return (
          <div key={index} style={containerStyle}>
            <div style={userContainer}>
              <div style={userStyle}>{each.question}</div>
            </div>
            <div style={agentContainer}>
              <div>
                <div style={answerContainer}>
                  <div style={answerLabel}>RAG Answer (from document):</div>
                  <div style={ragAnswerStyle}>{each.answer.ragAnswer}</div>
                </div>
                <div style={answerContainer}>
                  <div style={answerLabel}>MCP Answer (with web search):</div>
                  <div style={mcpAnswerStyle}>{each.answer.mcpAnswer}</div>
                </div>
              </div>
            </div>
          </div>
        );
      })}
      {isLoading && <Spin size="large" style={{ margin: "10px" }} />}
    </>
  );
};

export default RenderQA;
```

Last, let's pull everything together by updating **App.js**

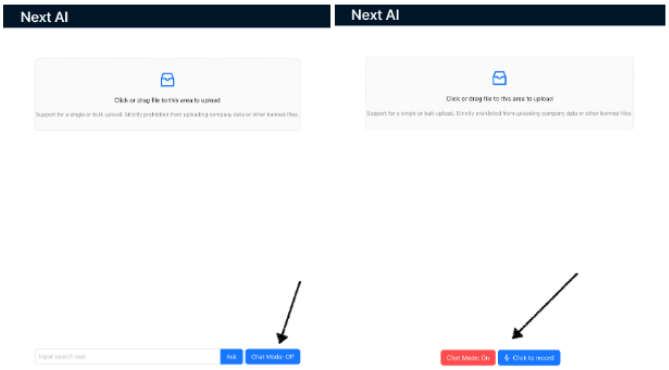
```
import React, { useState } from "react";
import PdfUploader from "./components/PdfUploader";
import ChatComponent from "./components/ChatComponent";
import RenderQA from "./components/RenderQA";
```

```
import { Layout, Typography } from "antd";

const chatComponentStyle = {
  position: "fixed",
  bottom: "0",
  width: "80%",
  left: "10%", // this will center it because it leaves 10% space on each side
  marginBottom: "20px",
};

const pdfUploaderStyle = {
  margin: "auto",
  paddingTop: "80px",
};

const renderQAStyle = {
  height: "50%", // adjust the height as you see fit
  overflowY: "auto",
};

const App = () => {
  const [conversation, setConversation] = useState([]);
  const [isLoading, setIsLoading] = useState(false);
  const { Header, Content } = Layout;
  const { Title } = Typography;

  const handleResp = (question, answer) => {
     setConversation((prev) => [...prev, { question, answer }]);
  };
```

```
  return (
    <>
      <Layout style={{ height: "100vh", backgroundColor: "white" }}>
        <Header
          style={{
            display: "flex",
            alignItems: "center",
          }}
        >
          <Title style={{ color: "white " }}>Agent AI</Title>
        </Header>
        <Content style={{ width: "80%", margin: "auto" }}>
          <div style={pdfUploaderStyle}>
            <PdfUploader />
          </div>

          <br />
          <br />
          <div style={renderQAStyle}>
            <RenderQA conversation={conversation} isLoading={isLoading} />
          </div>

          <br />
          <br />
        </Content>
        <div style={chatComponentStyle}>
          <ChatComponent
            handleResp={handleResp}
            isLoading={isLoading}
            setIsLoading={setIsLoading}
          />
        </div>
      </Layout>
    </>
  );
};

export default App;
```

Lesson 48: Add the Voice Interface

In this class, we will build the conversation user interface so can talk to the App using a natural language.



Add the Conversational Interface

First, we need to install a few dependencies

npm i react-speech-recognition@3.10.0
npm i speak-tts@2.0.8

Next, we will update the src/components/ChatComponent.js

```
import React, { useState, useEffect } from "react"; // Import useState
import axios from "axios";
import { Button, Input } from "antd";
import { AudioOutlined } from "@ant-design/icons";
import SpeechRecognition, {
  useSpeechRecognition,
} from "react-speech-recognition";
import Speech from "speak-tts";

const { Search } = Input;
```

offer版权所有

```
const DOMAIN = "http://localhost:5001";

const searchContainer = {
  display: "flex",
  justifyContent: "center",
};

const ChatComponent = (props) => {
  const { handleResp, isLoading, setIsLoading } = props;
  // Define a state variable to keep track of the search value
  const [searchValue, setSearchValue] = useState("");
  const [isChatModeOn, setIsChatModeOn] = useState(false);
  const [isRecording, setIsRecording] = useState(false);
  const [speech, setSpeech] = useState();

  // speech recognation
  const {
    transcript,
    listening,
    resetTranscript,
    browserSupportsSpeechRecognition,
    isMicrophoneAvailable,
  } = useSpeechRecognition();

  useEffect(() => {
    const initialized_speech = new Speech();
```

```
    initialized_speech
      .init({
        volume: 1,
        lang: "en-US",
        rate: 1,
        pitch: 1,
        voice: "Google US English",
        splitSentences: false,
      })
      .then((data) => {
        // The "data" object contains the list of available voices and the voice synthesis params
        console.log("Speech is ready, voices are available", data);
        setSpeech(initialized_speech);
      })
      .catch((e) => {
        console.error("An error occured while initializing : ", e);
      });
  }, []);

  useEffect(() => {
    if (!listening && Boolean(transcript)) {
      (async () => await onSearch(transcript))();
      setIsRecording(false);
    }
  }, [listening, transcript]);

  const talk = (what2say) => {
    speech
      .speak({
        text: what2say,
        queue: false, // current speech will be interrupted,
        listeners: {
          onstart: () => {
            console.log("Start utterance");
          },
          onend: () => {
            console.log("End utterance");
          },
          onresume: () => {
            console.log("Resume utterance");
          },
          onboundary: (event) => {
            console.log(
```

```
                event.name +
                  " boundary reached after " +
                  event.elapsedTime +
                  " milliseconds."
              );
            },
          },
        })
        .then(() => {
          // if everyting went well, start listening again
          console.log("Success !");
          userStartConvo();
        })
        .catch((e) => {
          console.error("An error occurred :", e);
        });
  };

  const userStartConvo = () => {
    SpeechRecognition.startListening();
    setIsRecording(true);
    resetTranscript();
  };

  const chatModeClickHandler = () => {
    setIsChatModeOn(!isChatModeOn);
```

```
    setIsRecording(false);
    SpeechRecognition.stopListening();

    resetTranscript();
  };

  const recordingClickHandler = () => {
    if (isRecording) {
      setIsRecording(false);
      SpeechRecognition.stopListening();

      resetTranscript();
    } else {
      setIsRecording(true);
      SpeechRecognition.startListening();
    }
  };
```

```
const onSearch = async (question) => {
  // Clear the search input
  setSearchValue("");
  setIsLoading(true);

  try {
    const response = await axios.get(`${DOMAIN}/chat`, {
      params: {
        question,
      },
    });
    handleResp(question, response.data);
    if (isChatModeOn) {
      talk(response.data?.ragAnswer);
    }
  } catch (error) {
    console.error(`Error: ${error}`);
    handleResp(question, error);
  } finally {
    setIsLoading(false);
  }
};

const handleChange = (e) => {
  // Update searchValue state when the user types in the input box
  setSearchValue(e.target.value);
};
```

Offer版权所有

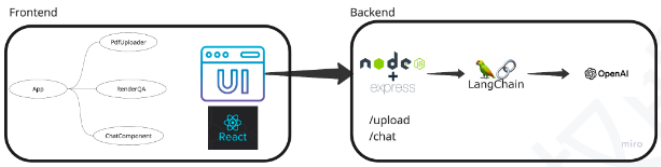
```
  return (
    <div style={searchContainer}>
      {!isChatModeOn && (
        <Search
          placeholder="input search text"
          enterButton="Ask"
          size="large"
          onSearch={onSearch}
          loading={isLoading}
          value={searchValue} // Control the value
          onChange={handleChange} // Update the value when changed
        />
      )}
      <Button
        type="primary"
        size="large"
        danger={isChatModeOn}
        onClick={chatModeClickHandler}
        style={{ marginLeft: "5px" }}
      >
        Chat Mode: {isChatModeOn ? "On" : "Off"}
      </Button>
      {isChatModeOn && (
```

```
        <Button
          type="primary"
          icon={<AudioOutlined />}
          size="large"
          danger={isRecording}
          onClick={recordingClickHandler}
          style={{ marginLeft: "5px" }}
        >
          {isRecording ? "Recording..." : "Click to record"}
        </Button>
      )}
    </div>
  );
};
```

```
export default ChatComponent;
```

That's it for the Agent AI Project. Now, have fun uploading documents and start chatting with the PDF!

High level Project Diagram



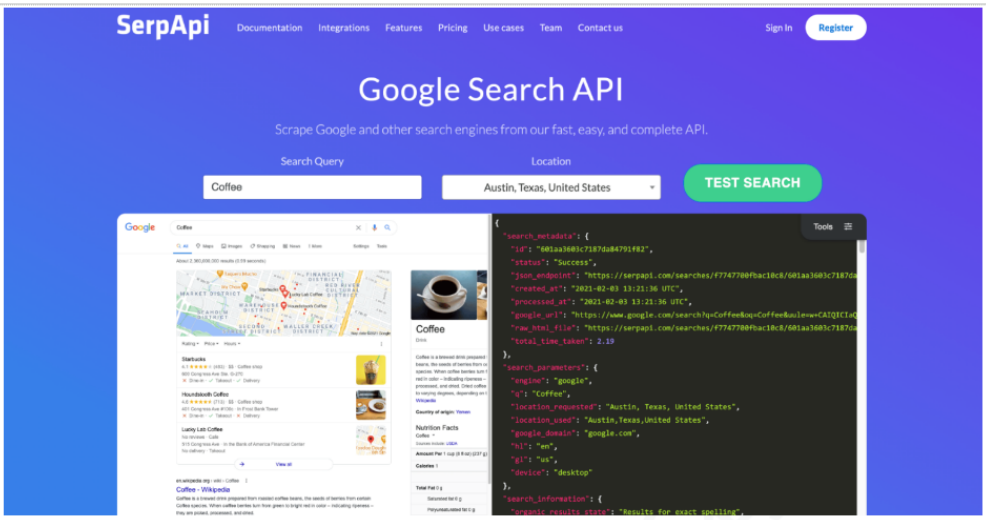
Non-Functional Requirements

- (optional) deploy the application to AWS/GCP
  - e.g. AWS Amplify (frontend), AWS App Runner (backend)
- consider non-functional requirements (NFR)
  - Performance - 性能
  - Scalability - 可扩展性
  - Reliability - 可靠性，正确性
  - Availability - 可用性，在线时间
  - Maintainability - 可维护性
  - Security - 安全性
  - Usability - 可用性 (UX)
  - Legal - 法律合规性
    - Accessibility (aria-label)

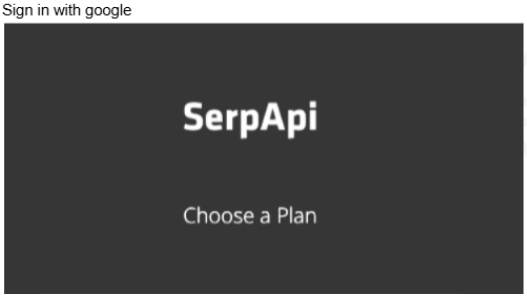
Next, let's get the serpAPI

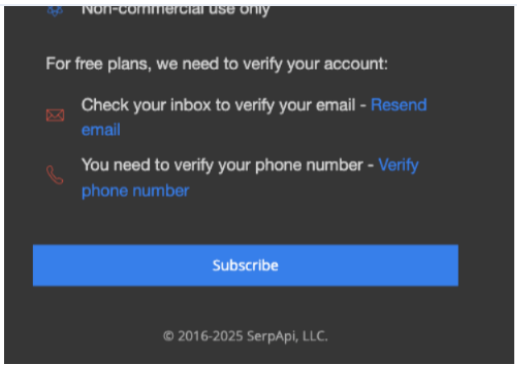
来Offer版权所有

Next, let's get the serpAPI

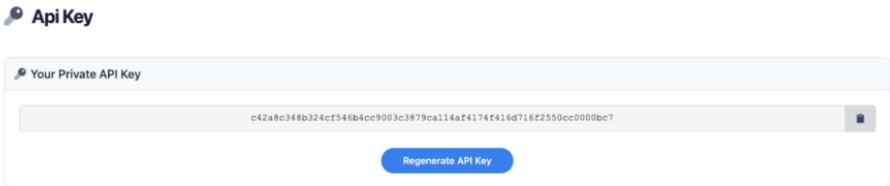


- go to https://serpapi.com/
- Sign in with google



- 

- follow the steps to verify, no credit card is needed, it's **FREE**.
- Click on Api Key on the left. Then get the key.



Open your **server/.env** file and add your key there

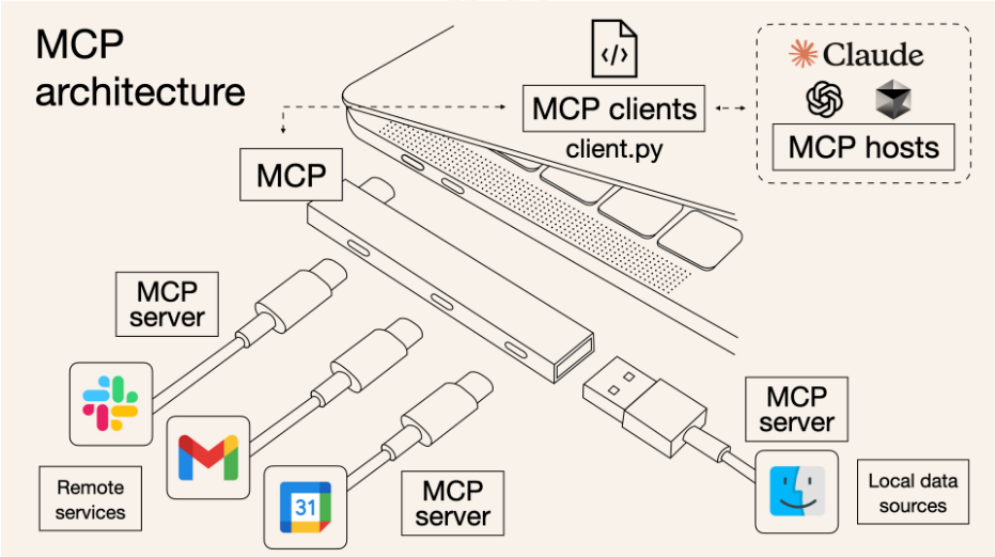
SERPAPI_KEY=<your_key>

Lesson 49: Model Context Protocol (MCP) Integration(LLM-based) AI Agents

1. has its objective
2. can interact with the outside world (through ***tool use***)
    a. change its own state
    b. change its environment's state
3. memory (short-term / long-term)

What is MCP → standardized tool use

Anthropic launched the Model Context Protocol (MCP) on **November 25, 2024**



Our architecture
HTTP(S) Request (React) → server.js → MCP Client (chat-mcp.js) → MCP Server (mcp-server.js, child process, Provides the search_web tool)

Let's create a file in server/mcp-server.js (MCP SERVER)

Let's create a file in server/mcp-server.js (MCP SERVER)

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
```

```
import { z } from "zod";
import { getJson } from "serpapi";

const SERPAPI_KEY = process.env.SERPAPI_KEY;

// Create MCP server instance
const server = new McpServer({
  name: "serpapi-search",
  version: "1.0.0",
});

// Register the search tool
server.registerTool(
  "search_web",
  {
    description:
      "Search the web using SerpAPI. Returns search results including organic results, snippets, and related
information.",
    inputSchema: {
      query: z.string().describe("The search query to execute"),
      num: z
        .number()
        .optional()
        .describe("Number of results to return (default: 10)"),
    },
  },
  async ({ query, num = 10 }) => {
    try {
      const results = await getJson({
        engine: "google",
        q: query,
        num: num,
        api_key: SERPAPI_KEY,
      });

      // Return all results as plain text
      const fullResults = JSON.stringify(results);
```

版权所有

```
      return {
        content: [
          {
            type: "text",
            text: fullResults,
          },
        ],
      };
    } catch (error) {
      return {
        content: [
          {
            type: "text",
            text: `Error performing web search: ${error.message}`,
          },
        ],
      };
    }
  }
);

// Main function to run the server
async function main() {
  const transport = new StdioServerTransport();
  await server.connect(transport);
  console.error("SerpAPI MCP Server running on stdio");
}

// Always run the server when this file is executed
// This is needed when spawned as a child process
main().catch((error) => {
```

```
  console.error("Fatal error in MCP server:", error);
  process.exit(1);
});

export default server;
```

Next, let's create server/chat-mcp.js **(MCP CLIENT)**

```
import { Client } from "@modelcontextprotocol/sdk/client/index.js";
import { StdioClientTransport } from "@modelcontextprotocol/sdk/client/stdio.js";
import { fileURLToPath } from "url";
import { dirname, join } from "path";
import { ChatOpenAI } from "@langchain/openai";
import { PromptTemplate } from "@langchain/core/prompts";

const __filename = fileURLToPath(import.meta.url);
const __dirname = dirname(__filename);

// Reusable MCP client instance
let client = null;
let transport = null;
let isConnecting = false;
let connectionPromise = null;

// Initialize the MCP client connection (lazy initialization)
const ensureConnected = async () => {
  // If already connected, return
  if (client && transport) {
    return;
  }

  // If connection is in progress, wait for it
  if (isConnecting && connectionPromise) {
    await connectionPromise;
    return;
  }

  // Start new connection
  isConnecting = true;
  connectionPromise = (async () => {
    try {
      // Create MCP client
      client = new Client({
        name: "chat-client",
        version: "1.0.0",
      });
```

来Offer版权所有

```
      // Get the path to the MCP server
      const serverPath = join(__dirname, "mcp-server.js");

      // Create transport to connect to the MCP server
      transport = new StdioClientTransport({
        command: "node",
        args: [serverPath],
        env: {
          ...process.env, // Inherit all environment variables from parent process
        },
      });

      // Connect to the MCP server
      await client.connect(transport);
    } finally {
      isConnecting = false;
      connectionPromise = null;
    }
  })();

  await connectionPromise;
};

const chatMCP = async (query) => {
  const apiKey = process.env.OPENAI_API_KEY;

  try {
    // Ensure client is connected (reuse existing connection)
    await ensureConnected();

    // Always perform web search
    const toolResult = await client.callTool({
      name: "search_web",
      arguments: {
        query: query,
        num: 5,
      },
    });

    let searchResults = "";
```

```
    if (toolResult.content && toolResult.content.length > 0) {
      searchResults = toolResult.content[0].text;
    }

    // Use the LLM to answer the question based on search results
    const model = new ChatOpenAI({
      model: "gpt-5",
      apiKey,
    });

    const answerTemplate = `Summarize the search result.

Search Results:
{searchResults}

Helpful Answer:`;

    const prompt = PromptTemplate.fromTemplate(answerTemplate);
    const formattedPrompt = await prompt.format({
      searchResults: searchResults || "No search results available",
    });

    const response = await model.invoke(formattedPrompt);
    const finalAnswer = response.content;

    return { text: finalAnswer };
  } catch (error) {
    // If connection error, reset client to allow reconnection on next request
    if (client) {
      try {
        await client.close();
      } catch (e) {
        // Ignore cleanup errors
      }
      client = null;
      transport = null;
    }
```

```
    throw error;
  }
};

export default chatMCP;
```

Now, last step, let's modify server.js

```
import express from "express";
import cors from "cors";
import dotenv from "dotenv";
import multer from "multer";
import chat from "./chat.js";
import chatMCP from "./chat-mcp.js";

dotenv.config();

const app = express();
app.use(cors());

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, "uploads/");
  },
  filename: (req, file, cb) => {
    cb(null, file.originalname);
  },
});

const upload = multer({
  storage,
});

const PORT = 5001;

let filePath;

app.post("/upload", upload.single("file"), (req, res) => {
  filePath = req.file.path;
```

```
app.post("/upload", upload.single("file"), (req, res) => {
  filePath = req.file.path;
  res.send(filePath + "uploaded successfully");
});

app.get("/chat", async (req, res) => {
  const ragResp = await chat(filePath, req.query.question);
  const mcpResp = await chatMCP(req.query.question);

  res.send({
    ragAnswer: ragResp.text,
    mcpAnswer: mcpResp.text,
  });
});

app.listen(PORT, () => {
  console.log("server is running on port " + PORT);
});
```

Great. Now you have everything and you can test it from web UI now!

Deployment.

- UI - AWS Amplify
- Servers - AWS App Runner

Lesson 50: React Summary

HTML

1. Q: Which tag is for hyperlinks in HTML, and what are the common attributes of this element ?

   A: <a> tag defines hyperlinks in HTML. In terms of attributes, “href” takes the value for the target web address, “target” specifies the browsing context, for instance, target=”_blank” will go to the link in a new browser tab.

2. Q: Block versus Inline element?

   A: Block-level elements form a visible block on a page. A block-level element appears on a new line following the content that precedes it. Any content that follows a block-level element also appears on a new line. Example - DIV

   Inline elements are contained within block-level elements, and surround only small parts of the document’s content. (not entire paragraphs or groupings of content) An inline element will not cause a new line to appear in the document. Example - Span

CSS

1. Q: Explain the conflicting rules in CSS

   A: **cascade** - at a very simple level this means that the order of CSS rules matter; when two rules apply that have equal specificity the one that comes last in the CSS is the one that will be used.

   Specificity - look up the specificity table, the selector with higher specificity value wins.

2. Q: What is the BOX Model in CSS?

   A: Box Model (content, padding, border, margin)

3. Q: What is the selector in CSS?

   A: Selector (type/class/ID, attribute, Pseudo-classes and pseudo-elements, Combinators)

Javascript

1. Q: How many different ways we have to define a variable in Javascript?

   A: Let, Const, Var (Describe the difference in these 3 categories: Scope, Reinitialization, Reference before declaration)

   https://www.freecodecamp.org/news/var-let-and-const-whats-the-difference/

2. Q: What is a callback function?

   A: A callback is a plain JavaScript function passed to some method as an argument or option. It is a function that is to be executed after another function has finished executing, hence the name 'call back'. In JavaScript, functions are objects. Because of this, functions can take functions as arguments, and can be returned by other functions.

3. Q: How do we get Data from the backend using JS usually?

   A: By making AJAX calls, nowadays the latest web api we use for this is Fetch API
   AJAX = Asynchronous JavaScript And XML

React

1. Q: Can you explain what virtual DOM is in React and how this technique can improve render performance?

   A: You tell React what state you want the UI to be in, and it makes sure the DOM matches that state. Representation of a UI is kept in memory and synced with the "real" DOM by a library - ReactDOM. React DOM compares the element and its children to the previous one, and only applies the DOM updates necessary to bring the DOM to the desired state.

   React - Virtual DOM

2. Q: What is the difference between props and state in React?

   A: Props are read-only, State is not read-only but you can't directly modify the State object, you will have to use the setState() method provided by React. Changes on either Props or State will trigger the re-render of the component.

3. Q: Can you list some common React Component Life Cycles and explain what they do?

   A: (Understand this diagram, focus on the common ones: didMount, didUpdate, willUnmount) React - Life-cycle Diagram

React

1. Q: Can you explain what virtual DOM is in React and how this technique can improve render performance?

   A: You tell React what state you want the UI to be in, and it makes sure the DOM matches that state. Representation of a UI is kept in memory and synced with the "real" DOM by a library - ReactDOM. React DOM compares the element and its children to the previous one, and only applies the DOM updates necessary to bring the DOM to the desired state.

   React - Virtual DOM

2. Q: What is the difference between props and state in React?

   A: Props are read-only, State is not read-only but you can't directly modify the State object, you will have to use the setState() method provided by React. Changes on either Props or State will trigger the re-render of the component.

3. Q: Can you list some common React Component Life Cycles and explain what they do?

   A: (Understand this diagram, focus on the common ones: didMount, didUpdate, willUnmount) React - Life-cycle Diagram

   React - Component Lifecycle

4. Q: Given this scenario, could you please implement this feature using React?

   - Clarify the requirements of this feature. What does the user need, how does each module interact with the user.
   - Design and think ALOUD! Tell the interviewer how you wanna design this UI, to be more specific, draw the React component hierarchy out and discuss with the interviewer to finalize what each component does.
   - Go for the implementation, don't panic, ask questions like "can i use **", "do we need to consider ***" whenever they come to you and block the implementation.

Typescript and Redux Intro

https://redux-toolkit.js.org/introduction/getting-started

```
npx create-react-app my-app --template cra-template-redux-typescript
```